

SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE:
CSA1304 COURSE OUTCOME COVERED

*CO4: Construct Context-Free Grammars, generate derivations
and parse trees to demonstrate the syntactic structure of strings.
(BL : 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:100 Duration : 1 Hour

Industry Problem

Code of Conduct:

*I Yuvann Nithish R (192411267) (Name / Reg No) certify that this
submission is my original work and that I have adhered to the guidelines
specified for this assessment.*

*I understand that any violation of academic integrity rules
will result in disciplinary action.*

Signature of the student: Yuvann Nithish R

SIMATS ENGINEERING ASSESSMENT TOOLS

1.Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

is the number of 0's in w

is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

SOLUTIONS:

Below are **exam/practical-ready solutions** for all 5 questions, with the **algorithm, DFA/PDA design, C code, and sample output**.

1. DFA – Strings Beginning with a and Ending with a

Language

[
L = {w \mid w \text{ begins with } a \text{ and ends with } a}
]

Examples:

a → Accepted

aba → Accepted

abba → Accepted

aa → Accepted

ab → Rejected

ba → Rejected

baba → Rejected

DFA States

q0 – Start state

q1 – String started with a and currently ends in a

q2 – String started with a but currently ends in b

qd – Dead/reject state

Transition Table

State a b

q0 q1 qd

q1 q1 q2

q2 q1 q2

qd qd qd

Final state: q1

C Program

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char input[100];
    int state = 0;

    printf("Enter a string containing a and b: ");
    scanf("%99s", input);

    for (int i = 0; input[i] != '\0'; i++) {

        char ch = input[i];

        if (state == 0) {
            if (ch == 'a')
                state = 1;
            else
                state = 3;
        }
    }
}
```

```

    }

    else if (state == 1) {
        if (ch == 'a')
            state = 1;
        else if (ch == 'b')
            state = 2;
        else
            state = 3;
    }

    else if (state == 2) {
        if (ch == 'a')
            state = 1;
        else if (ch == 'b')
            state = 2;
        else
            state = 3;
    }

    else {
        state = 3;
    }
}

if (state == 1)
    printf("Accepted\n");
else
    printf("Rejected\n");

return 0;
}

```

Sample

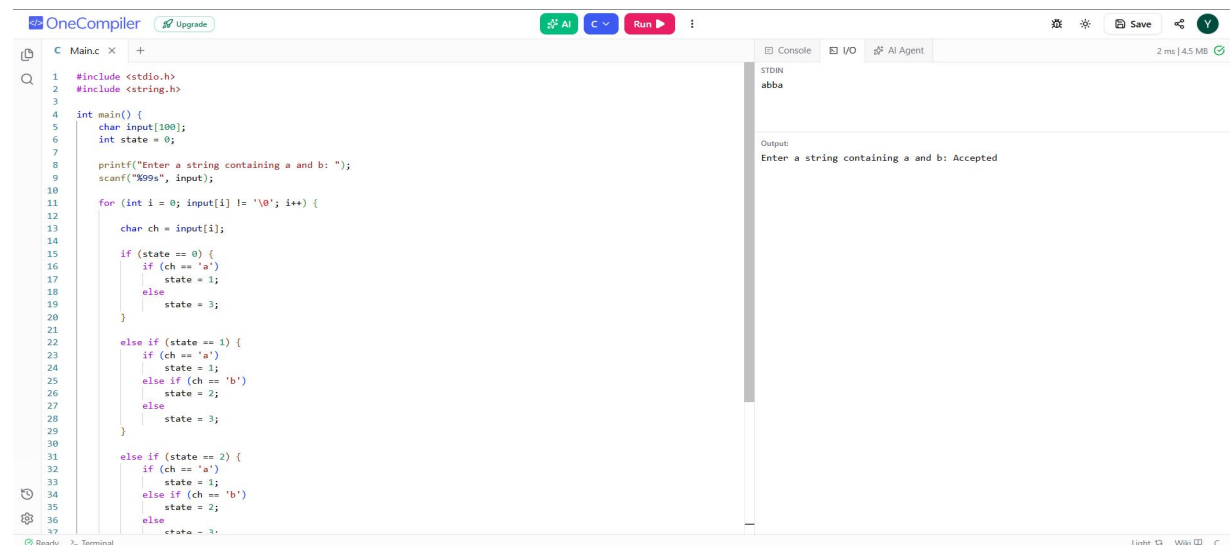
Enter a string containing a and b: abba

Accepted

Enter a string containing a and b: abb

Rejected

SAMPLE OUTPUT :



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char input[100];
6     int state = 0;
7
8     printf("Enter a string containing a and b: ");
9     scanf("%99s", input);
10
11     for (int i = 0; input[i] != '\0'; i++) {
12
13         char ch = input[i];
14
15         if (state == 0) {
16             if (ch == 'a')
17                 state = 1;
18             else
19                 state = 3;
20         }
21
22         else if (state == 1) {
23             if (ch == 'a')
24                 state = 1;
25             else if (ch == 'b')
26                 state = 2;
27             else
28                 state = 3;
29         }
30
31         else if (state == 2) {
32             if (ch == 'a')
33                 state = 1;
34             else if (ch == 'b')
35                 state = 2;
36             else
37                 state = 3;
38         }
39     }
40
41     if (state == 0 || state == 1 || state == 2)
42         printf("Accepted\n");
43     else
44         printf("Rejected\n");
45 }
```

STOR: abba

Output:
Enter a string containing a and b: Accepted

2. PDA – Equal Number of 0's and 1's

Your question appears to have lost the mathematical expression. I am assuming the intended language is:

$$L = \{w \mid \#0(w) = \#1(w)\}$$

That means the string must contain an **equal number of 0s and 1s**.

Examples:

01 → Accepted

10 → Accepted

0011 → Accepted

0101 → Accepted

0110 → Accepted

001 → Rejected

PDA Idea

Use the stack to keep track of the difference between the number of 0s and 1s.

When reading 0, push 0 if the stack represents excess 0s.

When reading 1, pop a 0 if possible.

Similarly, when reading 1 while there is excess 1s, push 1.

At the end, accept if only the bottom stack symbol remains.

PDA States

q_0 = processing input

q_f = final/accept state

Z_0 = bottom-of-stack symbol

Conceptual transitions

For a 0:

If top = 1 $\rightarrow$ pop 1

If top = Z_0 $\rightarrow$ push 0

If top = 0 $\rightarrow$ push 0

For a 1:

If top = 0 $\rightarrow$ pop 0

If top = Z_0 $\rightarrow$ push 1

If top = 1 $\rightarrow$ push 1

At the end:

$q_0, \epsilon, Z_0 \rightarrow q_f$

Example: 0011

Input	Stack

	Z0
0	0Z0
0	00Z0
1	0Z0
1	Z0

Stack returns to Z0.

Therefore:

0011 → **Accepted**

3. PDA – Language ($L = \{a^n b^{2n} \mid n \geq 1\}$)

We need:

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

Meaning:

Number of bs = twice the number of as.

Examples:

String	Number of a	Number of b	Result
abb	1	2	Accepted
aabbbb	2	4	Accepted
aaabbbbb	3	6	Accepted
ab	1	1	Rejected
aabbb	2	3	Rejected

PDA Design

For every a, push **two symbols** onto the stack.

Then, for every b, pop **one symbol**.

Therefore:

a $\rightarrow$ push XX

b $\rightarrow$ pop X

States

q0 – Reading as

q1 – Reading bs

qf – Accept state

Transitions

For every a:

[
 $\delta(q_0, a, Z) = (q_0, XXZ)$
]

and

[
 $\delta(q_0, a, X) = (q_0, XXX)$
]

Conceptually, each a adds **2 X's**.

When the first b occurs:

[
 $\delta(q_0, b, X) = (q_1, \epsilon)$
]

While reading b:

```
[  
\delta(q1,b,X)=(q1,\epsilon)  
]
```

When input finishes and stack reaches Z0:

```
[  
\delta(q1,\epsilon,Z0)=(qf,Z0)  
]
```

Example: aabbbb

Input	Stack
	Z0
a	XXZ0
a	XXXXZ0
b	XXXZ0
b	XXZ0
b	XZ0
b	Z0

Stack has returned to Z0.

Therefore:

aabbbb → Accepted

4. C Program – ϵ -Closure of All States in an NFA

What is ϵ -closure?

The **ϵ -closure of a state** is the set of all states that can be reached from that state using **zero or more ϵ -transitions**.

For example:

$q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2$

Then:

ϵ -closure(q_0) = { q_0, q_1, q_2 }

C Program

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int n;  
int epsilon[MAX][MAX];  
int closure[MAX][MAX];  
int visited[MAX];
```

```
void dfs(int start, int current) {  
  
    closure[start][current] = 1;  
    visited[current] = 1;  
  
    for (int i = 0; i < n; i++) {  
  
        if (epsilon[current][i] == 1 && !visited[i]) {  
            dfs(start, i);  
        }  
    }  
}
```

```
int main() {  
  
    printf("Enter number of states: ");  
    scanf("%d", &n);  
  
    printf("Enter epsilon transition matrix:\n");  
  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &epsilon[i][j]);  
        }  
    }  
  
    for (int i = 0; i < n; i++) {
```

```

    for (int j = 0; j < n; j++) {
        visited[j] = 0;
    }

    dfs(i, i);
}

printf("\nEpsilon Closure of all states:\n");

for (int i = 0; i < n; i++) {

    printf("ε-closure(q%d) = { ", i);

    for (int j = 0; j < n; j++) {

        if (closure[i][j])
            printf("q%d ", j);
    }

    printf("}\n");
}

return 0;
}

```

Sample Input

Suppose:

```

3
0 1 0
0 0 1
0 0 0

```

This represents:

```

q0 --ε--> q1
q1 --ε--> q2

```

Output

Epsilon Closure of all states:

ϵ -closure(q0) = { q0 q1 q2 }

$$\varepsilon\text{-closure}(q1) = \{ q1 \ q2 \}$$

$$\varepsilon\text{-closure}(q2) = \{ q2 \}$$

Important point

Every state is always included in its own ε -closure because **zero ε -transitions are allowed**.

SAMPLE OUTPUT:

```

1  #include <stdio.h>
2
3  #define MAX 20
4
5  int n;
6  int epsilon[MAX][MAX];
7  int closure[MAX][MAX];
8  int visited[MAX];
9
10 void dfs(int start, int current) {
11     closure[start][current] = 1;
12     visited[current] = 1;
13     for (int i = 0; i < n; i++) {
14         if (epsilon[current][i] == 1 && !visited[i]) {
15             dfs(start, i);
16         }
17     }
18 }
19
20 int main() {
21     printf("Enter number of states: ");
22     scanf("%d", &n);
23     printf("Enter epsilon transition matrix:\n");
24     for (int i = 0; i < n; i++) {
25         for (int j = 0; j < n; j++) {
26             scanf("%d", &epsilon[i][j]);
27         }
28     }
29     for (int i = 0; i < n; i++) {
30         dfs(i, i);
31     }
32     printf("Epsilon Closure of all states:\n");
33     for (int i = 0; i < n; i++) {
34         printf("epsilon closure(q%d) = {", i);
35         for (int j = 0; j < n; j++) {
36             if (closure[i][j] == 1) {
37                 printf("q%d ", j);
38             }
39         }
40         printf("}\n");
41     }
42 }

```

Output:

```

Enter number of states: Enter epsilon transition matrix:

Epsilon Closure of all states:
epsilon closure(q0) = { q0 q1 q2 q3 q4 }
epsilon closure(q1) = { q1 q2 q3 q4 }
epsilon closure(q2) = { q2 q3 q4 }
epsilon closure(q3) = { q3 q4 }
epsilon closure(q4) = { q4 }

```

5. C Program – CFG Validation

Given CFG

$$S \rightarrow 0A1$$

$$A \rightarrow 0A \mid 1A \mid \varepsilon$$

Understanding the Grammar

Start with:

$$S \rightarrow 0A1$$

So every valid string must:

Start with 0

End with 1

Have any combination of 0s and 1s in the middle

Therefore, the language is:

[
L = $\{0w1 \mid w \in \{0,1\}^*\}$
]

Examples

Input Result

01	Accepted
001	Accepted
011	Accepted
0001	Accepted
0101	Accepted
101	Rejected
010	Rejected
1	Rejected

C Program

```
#include <stdio.h>
#include <string.h>

int main() {

    char input[100];

    printf("Enter a binary string: ");
    scanf("%99s", input);

    int length = strlen(input);

    if (length < 2) {
        printf("Rejected\n");
        return 0;
    }
}
```

```

if (input[0] == '0' && input[length - 1] == '1') {

    int valid = 1;

    for (int i = 0; i < length; i++) {

        if (input[i] != '0' && input[i] != '1') {
            valid = 0;
            break;
        }
    }

    if (valid)
        printf("Accepted\n");
    else
        printf("Rejected\n");
}

else {
    printf("Rejected\n");
}

return 0;
}

```

Sample 1

Enter a binary string: 00101
Accepted

Why?

```

0 0 1 0 1
↑   ↑
start end
0   1

```

It satisfies $S \rightarrow 0A1$.

Sample 2

Enter a binary string: 1010
Rejected

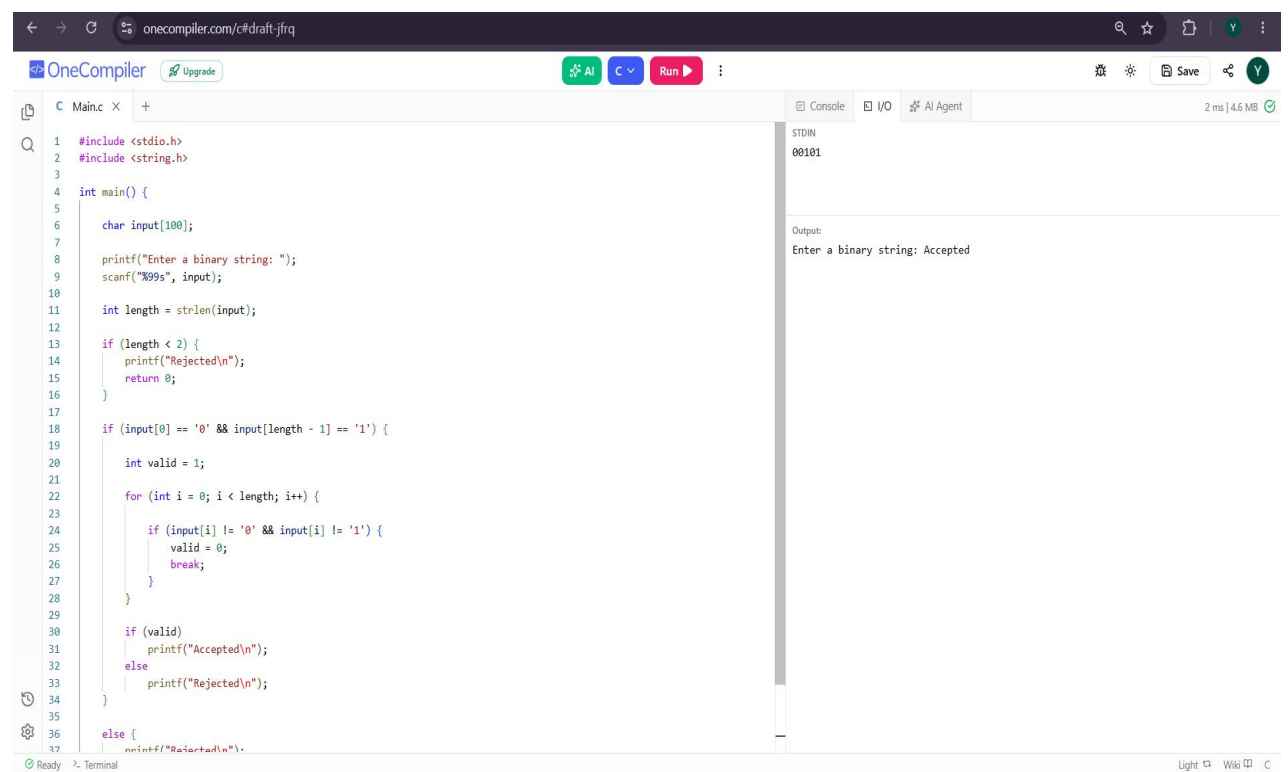
Why?

It starts with 1, but the grammar requires:

$S \rightarrow 0A1$

So it must start with 0 and end with 1.

SAMPLE OUTPUT:



The screenshot shows the OneCompiler website interface. The main editor displays a C program that checks if a binary string is accepted or rejected based on the grammar $S \rightarrow 0A1$. The program is run, and the output shows "Enter a binary string: Accepted".

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5
6     char input[100];
7
8     printf("Enter a binary string: ");
9     scanf("%99s", input);
10
11     int length = strlen(input);
12
13     if (length < 2) {
14         printf("Rejected\n");
15         return 0;
16     }
17
18     if (input[0] == '0' && input[length - 1] == '1') {
19
20         int valid = 1;
21
22         for (int i = 0; i < length; i++) {
23
24             if (input[i] != '0' && input[i] != '1') {
25                 valid = 0;
26                 break;
27             }
28         }
29
30         if (valid)
31             printf("Accepted\n");
32         else
33             printf("Rejected\n");
34     }
35
36     else {
37         printf("Rejected\n");
38     }
39 }
```

STDIN
00101

Output:
Enter a binary string: Accepted

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand